暑假考核

基本规范

- 使用 Git 管理代码。每完成一个阶段的题目，至少提交一次 commit，以体现代码版本管理能力。
- 编写 `README.md`，说明环境配置、项目编译及程序运行方法。
- 不强制规定代码组织形式，可根据自己的理解设计项目结构。
- 本次考核提供的示例代码可直接参考或使用。
- 加分项：合理使用开源项目，并明确标注项目来源。

Commit 规范

提交信息应包含类型（Type）和描述（Subject），也可以根据需要添加范围（Scope）、正文（Body）和脚注（Footer）。

```
<type>(<scope>): <subject>

<body>

<footer>
```

常用类型（Type）如下：

类型	描述
feat	新增功能（Feature）
fix	修复 Bug
docs	修改文档（Documentation）
style	修改代码格式，不影响程序逻辑，如调整空格、分号等
refactor	重构代码，既不修复 Bug，也不新增功能
perf	优化性能（Performance）
test	新增或修改测试用例
chore	调整构建流程、辅助工具或依赖库

考核题目

神经网络

任务目标

使用神经网络识别图像中的装甲板角点，并将识别结果可视化。

实现要求

1. 使用 YOLO26 训练一个姿态估计（Pose）模型。
2. 将训练完成的模型导出为 ONNX 格式，用于后续推理。
3. 使用题目提供的推理与可视化代码完成模型部署并跑通程序。

按照官方文档和默认参数独立完成模型训练即可。

- 官方文档
- 训练数据集
- 推理代码
- 测试视频（armors_yolo_test.mp4）

UART 串口通信

任务目标

运行附件程序 virtual_serial_tester。该程序会创建虚拟串口 `/tmp/test_com`，波特率为 `115200`。

读取并解析 `virtual_serial_tester` 发出的串口数据，将解析结果输出后，再按照相同协议重新打包并发送回去。

通信协议

```
字节偏移   内容                  说明
──────────────────────────────────────────
[0]       0xFF                 包头（固定）
[1]       uint8 值              单字节无符号整数
[2]       int16 高字节          大端序（高位在前）
[3]       int16 低字节          大端序
[4]       int16 低字节          小端序（低位在前）
[5]       int16 高字节          小端序
[6..14]   0x00                  填充字节（固定为全零）
[15]      0x0D                  包尾（固定）
```

> 本题主要考查二进制数据的理解，以及数据打包与解包能力。

ROS 2

读取装甲板测试视频，通过 ROS 2 发布图像话题，并使用 RViz2 查看该话题中的图像。

扩展卡尔曼滤波（EKF）

任务目标

在上一题的 ROS 2 项目基础上完成以下任务：

1. 识别视频中的装甲板。
2. 使用 PnP 算法解算装甲板姿态，获取其朝向角（Yaw）。
3. 使用 EKF 对机器人装甲板的 Yaw 观测值进行滤波。
4. 创建 ROS 2 自定义消息，同时发布观测 Yaw 和滤波后的 Yaw。

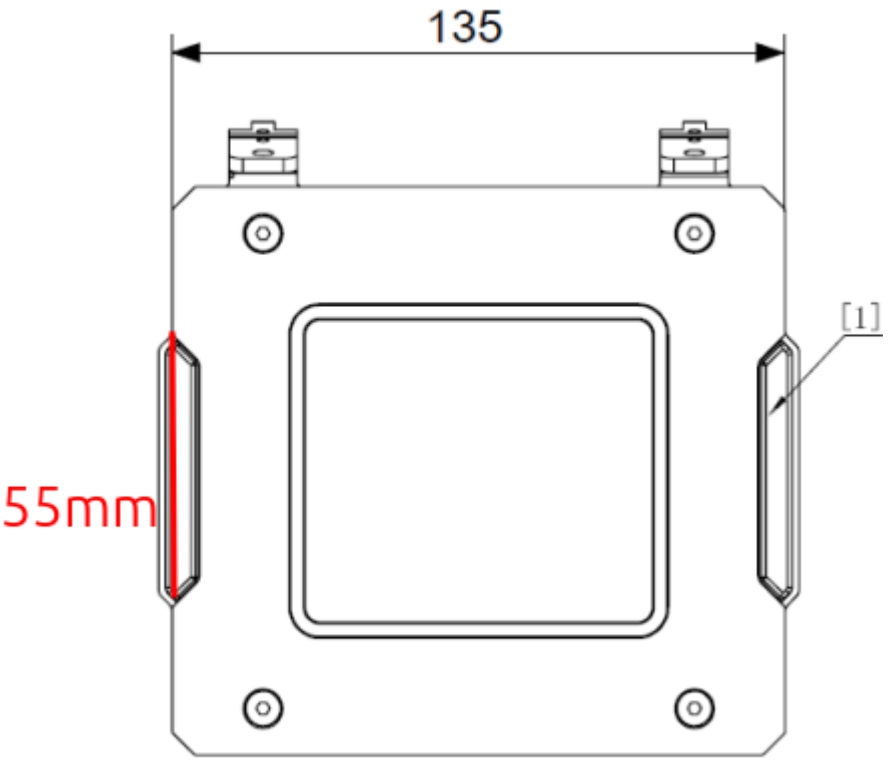
- EKF 示例代码
- 测试视频（test.mp4，后续题目均使用该视频）

> 本题默认使用神经网络进行装甲板识别。如果无法完成神经网络题目，可以申请放弃该题的神经网络部分并获取已训练模型，但不会获得对应的神经网络分数。
>
> 也可以使用寒假考核中的传统视觉方法识别装甲板，但同样不计入神经网络部分的分数。
>
> 以装甲板中心为原点

相机内参

```
image_width: 1440
image_height: 1080
camera_name: narrow_stereo
camera_matrix:
  rows: 3
  cols: 3
  data: [3571.1890927935297, 0.0, 683.5999418028833,
         0.0, 3569.0234714126823, 565.5838917436471,
         0.0, 0.0, 1.0]
distortion_model: plumb_bob
distortion_coefficients:
  rows: 1
  cols: 5
  data: [-0.029757126887136703, 0.371503828585697, 0.0011898367108863406,
-0.0025516396408119418, 0.0]
rectification_matrix:
  rows: 3
  cols: 3
  data: [1., 0., 0.,
         0., 1., 0.,
         0., 0., 1.]
projection_matrix:
  rows: 3
  cols: 4
  data: [3571.1890927935297,    0.     ,  683.5999418028833,    0.     ,
           0.     , 3569.0234714126823,  565.5838917436471,    0.     ,
           0.     ,    0.     ,    1.     ,    0.     ]
```

装甲板尺寸



坐标变换

任务目标

直接在相机坐标系下观测时，装甲板的 Yaw 可能变化不规律。这是因为装甲板主要相对于地面旋转，因此需要将姿态转换到世界坐标系（即里程计坐标系 `odom`）后再进行滤波。

将`cv::solvePnP`得到的`rvec`从相机坐标系转换到世界坐标系，提取转换后的 Yaw 并使用 EKF 进行滤波。创建 ROS 2 自定义消息，同时发布世界坐标系下的观测 Yaw 和滤波后的 Yaw。

坐标系关系

`cv::solvePnP`定义的相机系为`camera_optical_frame`坐标系。各坐标系的层级关系如下：

```
odom
 └── gimbal_link
       └── camera_link
            └── camera_optical_frame
```

坐标变换遵循以下约定：

- 变换顺序为先旋转、后平移。
- RPY（Roll、Pitch、Yaw）的旋转顺序为 Yaw、Pitch、Roll。
- Pitch 向下为正，Yaw 向左为正。
- 平移量`xyz`的单位为米（m），旋转量`rpy`的单位为弧度（rad）。

- 坐标系遵循 REP-103 规范，即 X 轴向前、Y 轴向左、Z 轴向上。

`odom` → `gimbal_link`

```
xyz = (0, 0, 0)
rpy = (0, 1.34, 0)
```

`gimbal_link` → `camera_link`

在 `gimbal_link` 坐标系下，先绕 Pitch 轴旋转 `0.091 rad`，再向前平移 `0.0461892 m`、向下平移 `0.0603874 m`，得到 `camera_link` 坐标系。

```
xyz = (0.0461892, 0, -0.0603874)
rpy = (0, 0.091, 0)
```

`camera_link` → `camera_optical_frame`

```
xyz = (0, 0, 0)
rpy = (-π/2, 0, -π/2)
```

重投影

任务目标

根据识别到的一块装甲板，推算机器人其余装甲板的位置，并将所有装甲板重投影到图像中进行可视化。

已知条件

已知装甲板在世界坐标系下的位置与姿态：

- 位置：(x_a, y_a, z_a)
- 姿态：$(yaw, pitch, roll)$

机器人中心位置为 (x_c, y_c, z_c)，装甲板与机器人中心满足：

$$x_a = x_c - \cos(yaw) \cdot r$$

$$y_a = y_c - \sin(yaw) \cdot r$$

默认参数如下：

$$r = 0.25, \quad z_a = z_c, \quad pitch = 15^\circ, \quad roll = 0$$

将第 i 块装甲板的 Yaw 表示为 $yaw + \frac{\pi}{2}i$，代入上述位置关系，即可计算其余装甲板的位置。

